Tether Backend Documentation

Version: 1.0.0
Last Updated: 2026
Project: Tether - Couples' Relationship App Backend

Table of Contents

Overview

Tether Backend is a RESTful API built with TypeScript and Express.js, designed to support a couples' relationship application. The backend provides comprehensive functionality for user management, couple pairing, personalized question delivery (tethers), subscription management, and push notifications.

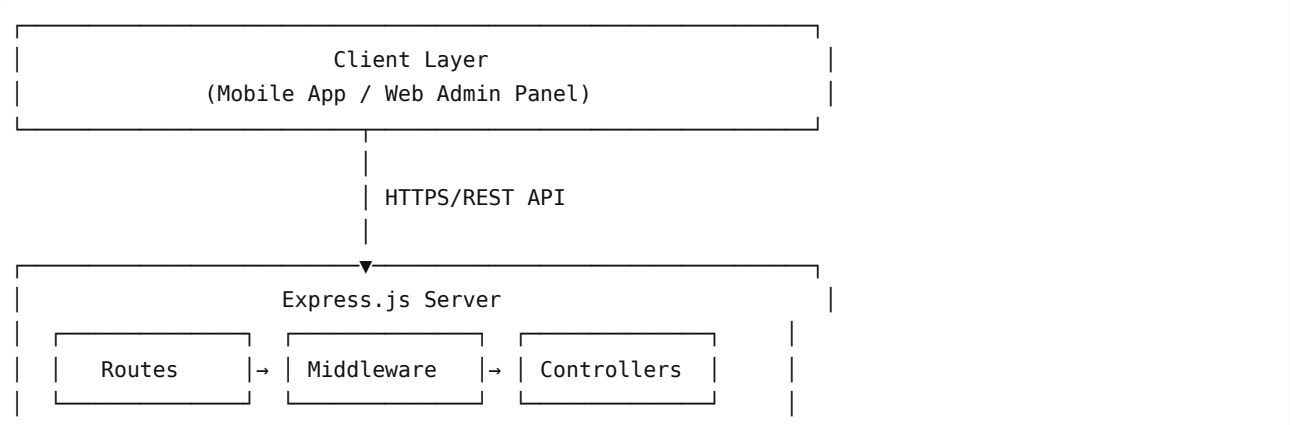
Key Features

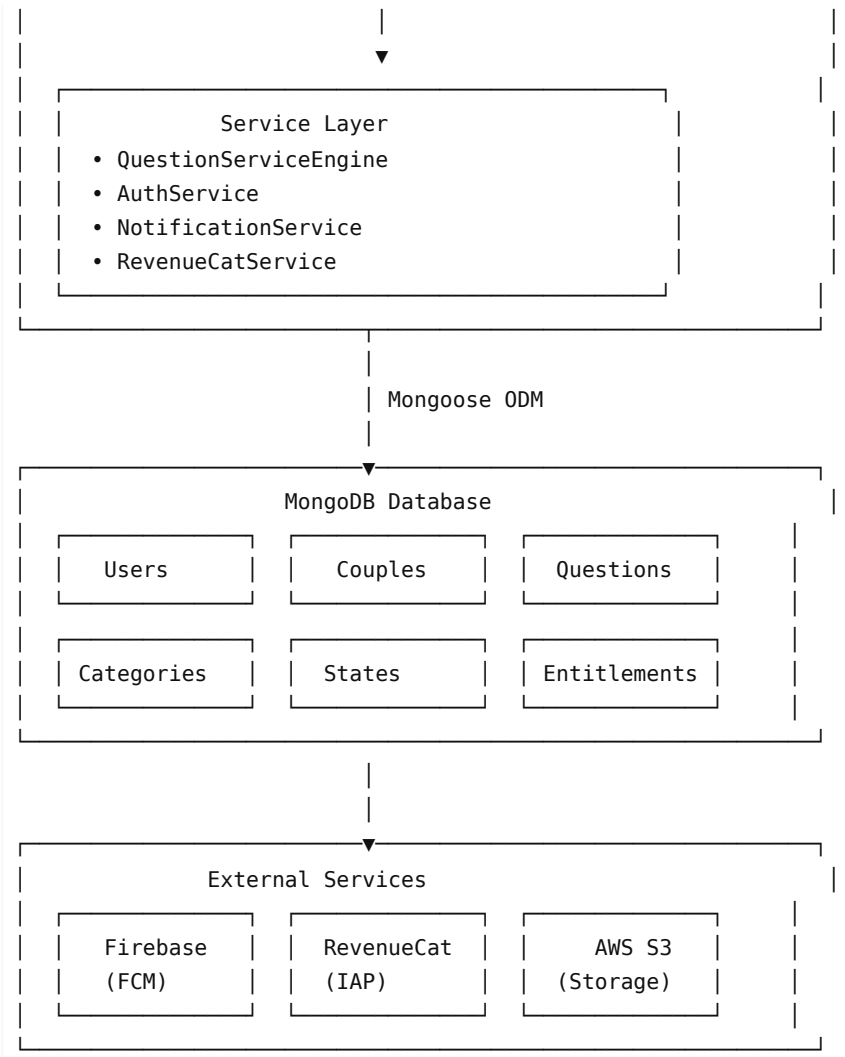
- **OAuth Authentication**: Google, Apple, and Email-based authentication
- **Couple Management**: Invite system and couple pairing
- **Question Service Engine**: Intelligent question delivery with personalization
- **Subscription Management**: Integration with RevenueCat for in-app purchases
- **Push Notifications**: Firebase Cloud Messaging (FCM) integration
- **Admin Panel**: Comprehensive admin interface for content management
- **Automated Scheduling**: Cron jobs for question drops and notifications

---

System Architecture

High-Level Architecture

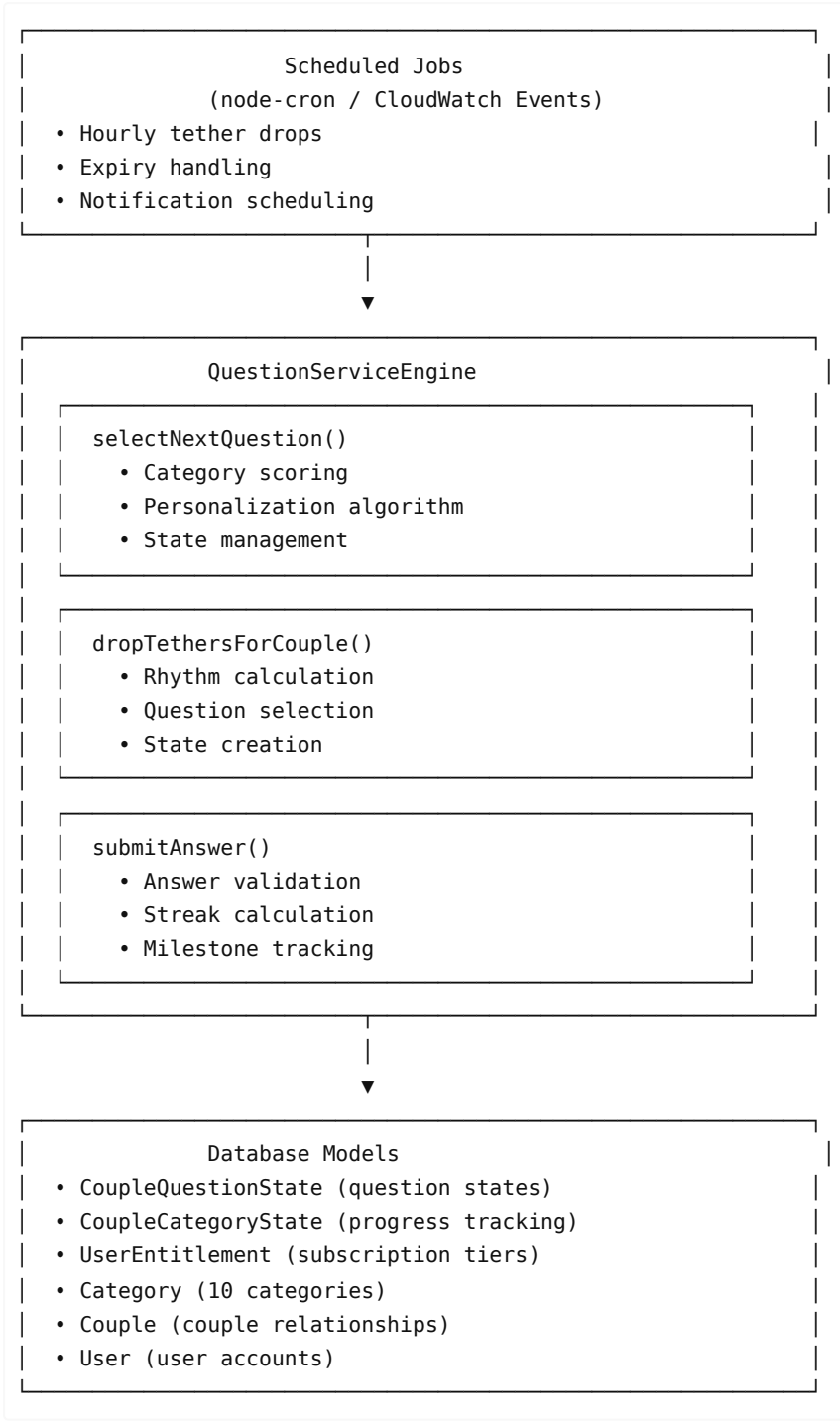




Request Flow Architecture

```
Client Request
    |
    ├─► Express Router (routes/index.ts)
    |       |
    |       ├─► Auth Middleware (JWT validation)
    |       |       |
    |       |       ├─► Admin Auth (for admin routes)
    |       |       └─► User Auth (for user routes)
    |       |
    |       └─► Controller Layer
    |               |
    |               ├─► Input Validation
    |               ├─► Business Logic (Service Layer)
    |               |       |
    |               |       ├─► Database Operations (Mongoose)
    |               |       ├─► External API Calls
    |               |       └─► Data Transformation
    |               |
    |               └─► Response (JSON)
```

Component Interaction Diagram



Technology Stack

Core Technologies

- **Runtime**: Node.js >= 18.0.0
- **Language**: TypeScript 5.9.3
- **Framework**: Express.js 4.22.1
- **Database**: MongoDB (via Mongoose 8.0.0)
- **Authentication**: JWT (jsonwebtoken 9.0.2)

Key Dependencies

Backend Framework

- `express` : Web framework

- `cors` : Cross-origin resource sharing
- `morgan` : HTTP request logger
- `dotenv` : Environment variable management

Database & ODM

- `mongoose` : MongoDB object modeling
- MongoDB connection via connection string

Authentication & Security

- `jsonwebtoken` : JWT token generation and verification
- `bcryptjs` : Password hashing
- `google-auth-library` : Google OAuth
- `apple-signin-auth` : Apple Sign In

File Handling

- `multer` : File upload middleware
- `@aws-sdk/client-s3` : AWS S3 integration for file storage

Scheduling & Automation

- `node-cron` : Cron job scheduling for automated tasks

External Services

- `firebase-admin` : Firebase Cloud Messaging (FCM)
- `axios` : HTTP client for external API calls
- `serverless-http` : Serverless deployment support

Documentation

- `swagger-jsdoc` : API documentation generation
- `swagger-ui-express` : Swagger UI interface

Utilities

- `uuid` : Unique identifier generation
- `xlsx` : Excel file processing

Development Tools

- `typescript` : TypeScript compiler
- `ts-node` : TypeScript execution
- `nodemon` : Development server with hot reload
- `eslint` : Code linting
- `prettier` : Code formatting
- `jest` : Testing framework

Deployment

- `serverless` : Serverless Framework for AWS Lambda
- `serverless-offline` : Local serverless development
- `serverless-dotenv-plugin` : Environment variable plugin

Project Structure

```
backend/
├── src/
│   ├── index.ts                    # Application entry point
│   │
│   ├── config/                     # Configuration files
│   │   └── swagger.ts              # Swagger/OpenAPI configuration
```

```
│   │
│   ├── controllers/               # Request handlers
│   │   ├── admin/                 # Admin controllers
│   │   │   ├── auth.ts            # Admin authentication
│   │   │   ├── category.ts        # Category management
│   │   │   ├── partners.ts        # Partner management
│   │   │   ├── question.ts        # Question CRUD
│   │   │   ├── stats.ts           # Statistics
│   │   │   ├── subscription.ts    # Subscription management
│   │   │   ├── unlocks.ts         # Category unlocks
│   │   │   └── users.ts           # User management
│   │   ├── auth.ts                # User authentication
│   │   ├── couple.ts              # Couple management
│   │   ├── notification.ts       # Notification handling
│   │   ├── onboarding.ts         # Onboarding flow
│   │   ├── profile.ts             # User profile
│   │   ├── revenuecat.ts         # RevenueCat webhooks
│   │   ├── settings.ts           # User settings
│   │   ├── subscription.ts       # Subscription endpoints
│   │   └── tether.ts             # Question service endpoints
│   │
│   ├── db/                        # Database configuration
│   │   └── db.ts                  # MongoDB connection
│   │
│   ├── middleware/                # Express middleware
│   │   └── auth.ts                # Authentication middleware
│   │
│   ├── models/                    # Mongoose schemas
│   │   ├── admin.ts              # Admin model
│   │   ├── Category.ts           # Category model
│   │   ├── CategoryProgress.ts   # Category progress (legacy)
│   │   ├── Couple.ts             # Couple model
│   │   ├── CoupleCategoryState.ts # Category state per couple
│   │   ├── CoupleInvite.ts       # Invite codes
│   │   ├── CoupleQuestionState.ts # Question state per couple
│   │   ├── Notification.ts       # Notification model
│   │   ├── Purchase.ts           # Purchase records
│   │   ├── Question.ts           # Question model
│   │   ├── Tether.ts             # Tether model (legacy)
│   │   ├── User.ts               # User model
│   │   └── UserEntitlement.ts    # Subscription entitlements
│   │
│   ├── routes/                    # API routes
│   │   ├── admin/                # Admin routes
│   │   │   ├── auth.ts
│   │   │   ├── category.ts
│   │   │   ├── partners.ts
│   │   │   ├── questions.ts
│   │   │   ├── stats.ts
│   │   │   ├── subscription.ts
│   │   │   ├── unlocks.ts
│   │   │   └── users.ts
│   │   ├── auth.ts               # Authentication routes
│   │   ├── couple.ts             # Couple routes
│   │   ├── index.ts              # Route aggregator
│   │   ├── logs.ts               # Logging routes
```

```
│   │   ├── notification.ts       # Notification routes
│   │   ├── onboarding.ts        # Onboarding routes
│   │   ├── profile.ts            # Profile routes
│   │   ├── revenuecat.ts         # RevenueCat routes
│   │   ├── settings.ts           # Settings routes
│   │   ├── subscription.ts       # Subscription routes
│   │   └── tethers.ts            # Tether/question routes
│   │
│   ├── services/                 # Business logic layer
│   │   ├── auth.ts               # Authentication service
│   │   ├── notification/         # Notification system
│   │   │   ├── notification.service.ts
│   │   │   ├── providers/
│   │   │   │   ├── expo.provider.ts
│   │   │   │   └── fcm.provider.ts
│   │   │   ├── scheduler.ts
│   │   │   └── triggers.ts
│   │   ├── questionService.ts    # Question Service Engine
│   │   ├── revenuecat/           # RevenueCat integration
│   │   │   └── revenuecat.service.ts
│   │   ├── scheduledJobs.ts      # Cron job definitions
│   │   └── user.ts               # User service
│   │
│   ├── scripts/                  # Utility scripts
│   │   ├── seedCategories.ts    # Seed categories
│   │   ├── seedQuestions.ts     # Seed questions
│   │   └── testCategoryScoring.ts
│   │
│   ├── types/                    # TypeScript definitions
│   │   ├── enums.ts             # Enumerations
│   │   ├── express.d.ts        # Express type extensions
│   │   ├── index.ts            # Type exports
│   │   └── interfaces.ts        # Interfaces
│   │
│   └── utils/                    # Utility functions
│       ├── index.ts             # General utilities
│       ├── logger.ts            # Logging utilities
│       ├── migrations.ts        # Database migrations
│       └── milestoneHelpers.ts  # Milestone calculations
│
├── config/                       # Configuration files
│   ├── aws/                     # AWS configurations
│   │   ├── dev-aws-config.json
│   │   ├── prod-aws-config.json
│   │   └── stage-aws-config.json
│   ├── env/                     # Environment variables
│   │   └── development.env
│   └── firebase-service-account.json
│
├── dist/                         # Compiled JavaScript (generated)
├── node_modules/                 # Dependencies
├── package.json                  # Project dependencies
├── tsconfig.json                 # TypeScript configuration
├── serverless.yml                # Serverless Framework config
└── README.md                     # Project README
```

Database Architecture

Database: MongoDB

The application uses MongoDB as the primary database, accessed through Mongoose ODM.

Collections & Models

1. Users Collection (`users`)

Model: `User`
Purpose: Stores user account information and authentication data

Key Fields:

- `_id` : ObjectId (primary key)
- `email` : String (unique, required)
- `name` : String
- `password` : String (hashed, optional for OAuth users)
- `provider` : Enum (GOOGLE, APPLE, EMAIL)
- `googleSub` : String (Google user ID, unique)
- `appleSub` : String (Apple user ID, unique)
- `avatar` : String (URL)
- `platform` : Enum (IOS, ANDROID, WEB)
- `onboarded` : Boolean
- `onboardingData` : Object
  - `firstName` : String
  - `partnerFirstName` : String
  - `dateOfBirth` : Date
  - `gender` : String
  - `relationshipStatus` : String
  - `relationshipDuration` : String
  - `livingType` : Array[String]
  - `hasChildren` : Boolean
  - `goals` : Array[String]
  - `emotionalNeeds` : Array[String]
  - `rhythm` : Enum
  - `tone` : Enum
  - `packPreferences` : Array[String]
- `coupleId` : ObjectId (reference to Couple)
- `fcmTokens` : Array[String]
- `apnsToken` : String
- `notificationPreferences` : Object
- `refreshTokens` : Array[String]
- `subscribed` : Boolean
- `createdAt` : Date
- `updatedAt` : Date

Indexes:

- `email` : Unique index
- `googleSub` : Sparse unique index
- `appleSub` : Sparse unique index
- `coupleId` : Index

2. Couples Collection (`couples`)

Model: `Couple`
Purpose: Represents a relationship between two users

Key Fields:

- `_id` : ObjectId (primary key)
- `user1Id` : ObjectId (reference to User)
- `user2Id` : ObjectId (reference to User)
- `status` : Enum (active, paused, ended)
- `rhythm` : Enum (EVERY_DAY, FEW_TIMES_WEEK, ONCE_WEEK, DECIDE_AS_GO)
- `lastTetherDrop` : Date
- `sharedData` : Object
  - `currentStreak` : Number
  - `totalTethersCompleted` : Number
  - `lastTetherDate` : Date
  - `milestoneRecords` : Array[{count, achievedAt, notified}]
  - `permanentRefreshBalance` : Number
- `createdAt` : Date
- `updatedAt` : Date

Indexes:

- `user1Id` : Index
- `user2Id` : Index
- Compound: `user1Id + user2Id`

3. Questions Collection (`questions`)

Model: `Question`
Purpose: Stores the question pool (1,800 questions)

Key Fields:

- `_id` : ObjectId (primary key)
- `text` : String (question text)
- `categoryId` : Enum (10 categories)
- `tags` : Array[String]
- `difficulty` : Enum (EASY, MEDIUM, HARD)
- `isActive` : Boolean
- `createdAt` : Date
- `updatedAt` : Date

Indexes:

- `categoryId` : Index
- `isActive` : Index
- Compound: `categoryId + isActive`

4. Categories Collection (`categories`)

Model: `Category`
Purpose: Defines the 10 question categories

Key Fields:

- `_id` : ObjectId (primary key)
- `categoryId` : Enum (unique identifier)
- `name` : String
- `description` : String
- `icon` : String
- `color` : String

- `order` : Number
- `isActive` : Boolean

Categories:

1. Communication
2. Intimacy
3. Trust
4. Playfulness
5. Vulnerability
6. Future
7. Gratitude
8. Conflict
9. Love Languages
10. Erotic

5. Couple Question States Collection (`couplequestionstates`)

Model: `CoupleQuestionState`
Purpose: Tracks question states per couple

Key Fields:

- `_id` : ObjectId (primary key)
- `coupleId` : ObjectId (reference to Couple)
- `questionId` : ObjectId (reference to Question)
- `categoryId` : Enum
- `state` : Enum (PENDING, ANSWERED, SKIPPED, EXPIRED)
- `droppedAt` : Date
- `answeredAt` : Date
- `expiresAt` : Date
- `user1Answer` : String
- `user2Answer` : String
- `user1AnsweredAt` : Date
- `user2AnsweredAt` : Date

Indexes:

- `coupleId` : Index
- `questionId` : Index
- `state` : Index
- `expiresAt` : Index
- Compound: `coupleId + state`
- Compound: `coupleId + categoryId`

6. Couple Category States Collection (`couplecategorystates`)

Model: `CoupleCategoryState`
Purpose: Tracks category progress per couple

Key Fields:

- `_id` : ObjectId (primary key)
- `coupleId` : ObjectId (reference to Couple)
- `categoryId` : Enum
- `isUnlocked` : Boolean
- `questionsAnswered` : Number
- `lastQuestionAt` : Date
- `unlockedAt` : Date

Indexes:

- `coupleId` : Index
- `categoryId` : Index
- Compound: `coupleId + categoryId` (unique)

7. User Entitlements Collection (`userentitlements`)

Model: `UserEntitlement`
Purpose: Manages subscription tiers and refresh balances

Key Fields:

- `_id` : ObjectId (primary key)
- `userId` : ObjectId (reference to User, unique)
- `tier` : Enum (FREE, TRIAL, PREMIUM)
- `refreshesDefault` : Number (tier-based refreshes)
- `refreshesPermanent` : Number (purchased refreshes)
- `trialEnd` : Date (for TRIAL tier)
- `createdAt` : Date
- `updatedAt` : Date

Indexes:

- `userId` : Unique index

8. Couple Invites Collection (`coupleinvites`)

Model: `CoupleInvite`
Purpose: Manages couple invitation codes

Key Fields:

- `_id` : ObjectId (primary key)
- `inviterId` : ObjectId (reference to User)
- `inviteCode` : String (6-digit code, unique)
- `inviteLink` : String
- `status` : Enum (pending, accepted, expired)
- `expiresAt` : Date
- `acceptedById` : ObjectId
- `acceptedAt` : Date
- `createdAt` : Date

Indexes:

- `inviteCode` : Unique index
- `inviterId` : Index
- `status` : Index

9. Purchases Collection (`purchases`)

Model: `Purchase`
Purpose: Records in-app purchase transactions

Key Fields:

- `_id` : ObjectId (primary key)
- `userId` : ObjectId (reference to User)
- `productId` : String
- `transactionId` : String
- `platform` : Enum (IOS, ANDROID)
- `status` : Enum
- `amount` : Number
- `currency` : String

- `purchasedAt` : Date

Indexes:

- `userId` : Index
- `transactionId` : Unique index

10. Notifications Collection (`notifications`)

Model: `Notification`
Purpose: Stores notification records

Key Fields:

- `_id` : ObjectId (primary key)
- `userId` : ObjectId (reference to User)
- `type` : Enum
- `title` : String
- `body` : String
- `data` : Object
- `read` : Boolean
- `sentAt` : Date
- `readAt` : Date

Indexes:

- `userId` : Index
- `read` : Index
- Compound: `userId + read`

11. Admins Collection (`admins`)

Model: `Admin`
Purpose: Admin user accounts

Key Fields:

- `_id` : ObjectId (primary key)
- `email` : String (unique)
- `password` : String (hashed)
- `role` : Enum (admin, super_admin)
- `isActive` : Boolean
- `createdAt` : Date

Indexes:

- `email` : Unique index

Database Relationships

```
User (1) ──────── (1) Couple (1) ──────── (1) User
  │                    │
  │                    ├──► CoupleQuestionState (many)
  │                    ├──► CoupleCategoryState (many)
  │                    └──► CoupleInvite (many)
  │
  ├──► UserEntitlement (1)
  ├──► Purchase (many)
  └──► Notification (many)
```

```
Question (many) ──► CoupleQuestionState (many)
Category (many) ──► CoupleCategoryState (many)
```

API Endpoints

Base URL

- Development: `http://localhost:3000`
- Production: (configured per environment)

Authentication

All protected endpoints require a JWT token in the Authorization header:

```
Authorization: Bearer <token>
```

Public Endpoints

Health Check

```
GET /api/health
```

Returns server status and timestamp.

Authentication

```
POST /api/auth/google
POST /api/auth/apple
POST /api/auth/email/register
POST /api/auth/email/login
POST /api/auth/refresh
```

Protected User Endpoints

Onboarding

```
POST /api/onboarding/update
POST /api/onboarding/complete
POST /api/onboarding/invite/generate
POST /api/onboarding/invite/accept
GET  /api/onboarding/couple
```

Couple Management

```
GET  /api/couple
POST /api/couple/invite
POST /api/couple/join
```

Question Service (Tethers)

```
GET  /api/tethers/active
POST /api/tethers/answer
POST /api/tethers/skip
GET  /api/tethers/categories/progress
POST /api/tethers/categories/unlock
POST /api/tethers/drop
```

```
GET  /api/tethers/stats
POST /api/tethers/initialize
```

Profile & Settings

```
GET  /api/profile
PUT  /api/profile
GET  /api/settings
PUT  /api/settings
```

Subscription

```
GET  /api/subscription/status
POST /api/subscription/verify
```

Notifications

```
GET  /api/notifications
PUT  /api/notifications/:id/read
POST /api/notifications/preferences
```

Admin Endpoints

All admin endpoints require admin authentication.

Admin Authentication

```
POST /api/admin/auth/login
POST /api/admin/auth/refresh
```

Question Management

```
GET    /api/admin/questions
POST   /api/admin/questions
PUT    /api/admin/questions/:id
DELETE /api/admin/questions/:id
```

Category Management

```
GET    /api/admin/categories
POST   /api/admin/categories
PUT    /api/admin/categories/:id
DELETE /api/admin/categories/:id
```

User Management

```
GET    /api/admin/users
GET    /api/admin/users/:id
PUT    /api/admin/users/:id
DELETE /api/admin/users/:id
```

Statistics

```
GET /api/admin/stats
GET /api/admin/stats/users
GET /api/admin/stats/couples
GET /api/admin/stats/questions
```

RevenueCat Webhooks

```
POST /api/revenuecat/webhook
```

Core Services

1. Question Service Engine

File: `src/services/questionService.ts`
Purpose: Core engine for question delivery and personalization

Key Methods:

- `selectNextQuestion(coupleId, categoryId)` : Selects the next question based on personalization algorithm
- `dropTethersForCouple(coupleId, force)` : Drops new questions for a couple
- `submitAnswer(coupleId, questionStateId, userId, answer)` : Processes answer submission
- `skipQuestion(coupleId, questionStateId, userId)` : Handles question skipping with refresh logic
- `initializeCategoriesForCouple(coupleId)` : Initializes category states for a new couple
- `calculateCategoryScore(categoryId, coupleProfile)` : Calculates relevance score for categories

Personalization Algorithm:

- Uses weighted scoring based on onboarding data (goals, emotional needs, living type)
- Considers question history and cooldown periods
- Respects subscription tier (FREE: 2 categories, PREMIUM: 10 categories)

2. Authentication Service

File: `src/services/auth.ts`
Purpose: Handles authentication logic

Features:

- JWT token generation (access + refresh tokens)
- OAuth integration (Google, Apple)
- Email/password authentication
- Token refresh mechanism
- Password hashing with bcrypt

3. Notification Service

File: `src/services/notification/notification.service.ts`
Purpose: Manages push notifications

Providers:

- Firebase Cloud Messaging (FCM)
- Expo Push Notifications

Features:

- Scheduled notifications
- Notification triggers (couple created, tether dropped, milestone achieved)

- Notification preferences management

4. RevenueCat Service

File: `src/services/revenuecat/revenuecat.service.ts`
Purpose: Integrates with RevenueCat for subscription management

Features:

- Webhook handling
- Subscription status verification
- Entitlement management

5. Scheduled Jobs

File: `src/services/scheduledJobs.ts`
Purpose: Automated background tasks

Jobs:

- Hourly tether drops for active couples
- Expiry handling for unanswered questions
- Notification scheduling

Authentication & Authorization

Authentication Flow

1. **User Registration/Login**
   - User provides credentials (OAuth or email/password)
   - Server validates credentials
   - Server generates JWT access token and refresh token
   - Tokens returned to client
2. **Token Usage**
   - Client includes access token in `Authorization: Bearer <token>` header
   - Middleware validates token
   - Request proceeds if valid
3. **Token Refresh**
   - When access token expires, client uses refresh token
   - Server validates refresh token
   - New access token issued

Middleware

User Authentication (`authMiddleware`)

- Validates JWT token
- Extracts user ID from token
- Fetches user from database
- Attaches user to request object
- Used for all user-facing endpoints

Admin Authentication (`adminAuth`)

- Validates JWT token with `type: 'admin'`
- Verifies admin exists and is active
- Attaches admin to request object

- Used for all admin endpoints

Super Admin (`superAdminAuth`)

- Requires admin authentication first
- Verifies admin role is `super_admin`
- Used for sensitive admin operations

Token Structure

Access Token:

```
{
  "userId": "user_id_here",
  "type": "user",
  "tokenType": "access",
  "iat": 1234567890,
  "exp": 1234571490
}
```

Refresh Token:

```
{
  "userId": "user_id_here",
  "type": "user",
  "tokenType": "refresh",
  "iat": 1234567890,
  "exp": 1234654290
}
```

Question Service Engine

Overview

The Question Service Engine is the core system that delivers personalized questions (tethers) to couples. It manages question selection, state tracking, and engagement metrics.

Key Concepts

Categories

- 10 predefined categories covering different relationship aspects
- FREE tier: Access to 2 categories
- PREMIUM tier: Access to all 10 categories

Question States

- `PENDING` : Question dropped, waiting for answers
- `ANSWERED` : Both partners answered
- `SKIPPED` : Question skipped (uses refresh)
- `EXPIRED` : Question expired without answers

Subscription Tiers

- `FREE` : 1 refresh per day, 2 categories
- `TRIAL` : 3 refreshes per day, all categories (time-limited)
- `PREMIUM` : 3 refreshes per day, all categories

Personalization Algorithm

1. **Category Scoring**
    - Analyzes couple's onboarding data (goals, emotional needs, living type)
    - Calculates relevance score for each category
    - Prioritizes categories with higher scores
2. **Question Selection**
    - Considers question history (cooldown period: 14 days)
    - Avoids recently answered questions
    - Selects from unlocked categories
    - Respects subscription tier
3. **Rhythm Management**
    - Drops questions based on couple's rhythm preference
    - Tracks `lastTetherDrop` to prevent over-dropping
    - Supports: Every Day, Few Times Week, Once Week, Decide As Go

Automated Features

Scheduled Drops

- Runs hourly via cron job
- Checks all active couples
- Drops new questions if rhythm interval passed
- Handles expiry of unanswered questions

Streak Tracking

- Tracks consecutive days with completed tethers
- Resets if a day is missed
- Updates `currentStreak` in couple's shared data

Milestone System

- Tracks total tethers completed: 5, 10, 25, 50, 100
- Sends notifications on milestone achievement
- Records milestone history

Notification System

Architecture

```
Notification Service
    │
    ├──► Scheduler (node-cron)
    │       │
    │       └──► Triggers
    │               ├──► onCoupleCreated()
    │               ├──► onTetherDropped()
    │               ├──► onMilestoneAchieved()
    │               └──► onTetherExpiring()
    │
    └──► Providers
            ├──► FCM Provider (Firebase)
            └──► Expo Provider
```

Notification Types

- **Couple Created**: Sent when couple is formed

- **Tether Dropped**: New question available
- **Tether Expiring**: Reminder before question expires
- **Milestone Achieved**: Streak or completion milestone
- **Partner Answered**: Notification when partner answers

Configuration

Notifications are configured via:

- User notification preferences
- Firebase service account credentials
- Scheduled job intervals

Deployment

Serverless Deployment (AWS Lambda)

The application is configured for serverless deployment using the Serverless Framework.

Configuration File: `serverless.yml`

Key Settings:

- Runtime: Node.js 20.x
- Region: us-east-1
- Memory: 512 MB
- Timeout: 29 seconds
- Handler: `dist/index.handler`

Deployment Commands

```
# Deploy to staging
serverless deploy --stage staging

# Deploy to production
serverless deploy --stage production
```

Environment Variables

Set via Serverless Framework environment configuration or AWS Systems Manager Parameter Store.

Traditional Server Deployment

For traditional server deployment:

```
# Build
npm run build

# Start
NODE_ENV=production npm start
```

Scheduled Jobs

Serverless: Use AWS CloudWatch Events to trigger Lambda functions for scheduled tasks.

Traditional Server: Use `node-cron` (already configured in `scheduledJobs.ts`).

Environment Configuration

Environment Files

Located in `config/env/`:

- `development.env`: Development environment
- `stage.env`: Staging environment
- `prod.env`: Production environment

Required Environment Variables

```
# Database
DATABASE_URL=mongodb://localhost:27017/tether

# Server
PORT=3000
NODE_ENV=development

# JWT
JWT_SECRET=your-secret-key
JWT_REFRESH_SECRET=your-refresh-secret
JWT_EXPIRES_IN=1h
JWT_REFRESH_EXPIRES_IN=7d

# OAuth
GOOGLE_CLIENT_ID=your-google-client-id
GOOGLE_CLIENT_SECRET=your-google-client-secret
APPLE_CLIENT_ID=your-apple-client-id

# AWS
AWS_REGION=us-east-1
AWS_ACCESS_KEY_ID=your-access-key
AWS_SECRET_ACCESS_KEY=your-secret-key
S3_BUCKET=your-bucket-name

# Firebase (FCM)
FIREBASE_SERVICE_ACCOUNT_PATH=./config/firebase-service-account.json
# OR
FIREBASE_SERVICE_ACCOUNT={"type":"service_account",...}

# RevenueCat
REVENUECAT_API_KEY=your-revenuecat-api-key
REVENUECAT_SANDBOX=true

# CORS (comma-separated)
ALLOWED_ORIGINS=http://localhost:5173,https://yourdomain.com
```

AWS Configuration

AWS-specific configurations in `config/aws/`:

- `dev-aws-config.json`
- `stage-aws-config.json`
- `prod-aws-config.json`

Development Guidelines

Code Style

- **TypeScript**: Strict mode enabled
- **ESLint**: Enforces code standards
- **Prettier**: Consistent formatting
- **Conventional Commits**: Commit message format

Type Safety

- Avoid `any` types
- Use interfaces and types from `src/types/`
- Define proper return types for functions

Error Handling

- Use try-catch blocks for async operations
- Return appropriate HTTP status codes
- Provide meaningful error messages
- Log errors for debugging

Testing

- Write unit tests for services
- Test API endpoints with integration tests
- Use Jest testing framework

Git Workflow

1. Create feature branch: `git checkout -b feature/your-feature`
2. Make changes
3. Run linting: `npm run lint`
4. Run tests: `npm test`
5. Commit with conventional commits
6. Create pull request

API Documentation

- Swagger UI available at `/api-docs`
- Document endpoints with JSDoc comments
- Keep API documentation up to date

Conclusion

This documentation provides a comprehensive overview of the Tether Backend architecture, components, and development guidelines. For specific implementation details, refer to the source code and inline comments.

For questions or support, refer to the project README or contact the development team.

Document Version: 1.0.0
Last Updated: 2026
Maintained by: Tether Development Team